Ecole Supérieure d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

RAPPORT DU PROJET

4-ème année

CYBERSECURITY

Dans le cadre du cours :

Parallel Programing

Sous le thème :

Implémentation et optimisation RSA et Factorisation de Fermat

Réalisé par : Ayman bouissehak **Encadré par : Mohamed Bakhouya**

Année Universitaire : 2025-2026

Table des matières

Introduction 2

Partie 1 – Implémentation RSA 3

1. Algorithmes et principes : 3
2. Génération des clés : 4
3. Validation par chiffrement et déchiffrement 5
4. Comparaison CPU / GPU (RSA) 5

Partie 2 – Factorisation de Fermat 7

1. Algorithme de Fermat 7

Étapes de l'algorithme 7

2. Implémentations séquentielles et parallèles 8

2.1 Version séquentielle 8

2.2 Version MPI 8

2.3 Version UPC 9

2.4 Version CUDA 9

3. Comparaison des performances et speedup 10

a) Temps d'exécution (ms) : 10

b) Accélération (Speedup) : 11

c) Efficacité : 11

d) Tableau récapitulatif : 11

Conclusion 12

Introduction

La sécurité des systèmes informatiques repose largement sur des algorithmes cryptographiques efficaces et robustes. Le RSA est l'un des systèmes de chiffrement asymétrique les plus utilisés, reposant sur la difficulté de factoriser de grands nombres semi-premiers.

Ce projet vise à implémenter et tester l'algorithme RSA, ainsi que l'algorithme de factorisation de Fermat pour retrouver les facteurs premiers d'un nombre semi-premier.

Nous avons exploré différentes stratégies d'optimisation et de parallélisation afin de comparer les performances entre des versions séquentielles et des versions parallèles utilisant MPI, UPC et CUDA.

L'objectif principal est de mesurer l'impact de la parallélisation sur le temps d'exécution et d'analyser les gains obtenus sur CPU et GPU.

Partie 1 – Implémentation RSA

1. Algorithmes et principes :

RSA est un système de chiffrement asymétrique.

Il repose sur le produit de deux grands nombres premiers $p$et $q$pour créer un nombre semi-premier $n = p \times q$.

Les étapes principales sont :

Choisir deux nombres premiers $p$et q.

Calculer $n = p \cdot q$et $\phi(n) = (p-1)(q-1)$.

Choisir un exposant public $e$tel que $1 < e < \phi(n)$et $\gcd(e, \phi(n)) = 1$.

Calculer l'exposant privé $d$tel que $d \cdot e \equiv 1 \mathrm{mod}\ \phi(n)$(avec l'**algorithme d'Euclide étendu**).

Formules principales :

$$n = p \cdot q$$

$$\phi(n) = (p-1) \cdot (q-1)$$

$$d \cdot e \equiv 1 \pmod{\phi(n)}$$

Le **message M** est converti en entier.

Le **chiffrement** : $C \equiv M^e \mathrm{mod}\ n$

Le **déchiffrement** : $M \equiv C^d \mathrm{mod}\ n$

L'implémentation peut utiliser **GMP pour les grands entiers** ou des types natifs 64 bits pour des tests plus simples.

2. Génération des clés :

Génération de nombres premiers **aléatoires** pour $p$et q.

Calcul de $n$et $\phi(n)$.

Choix de e(souvent 65537) et calcul de $d$avec **Extended Euclid**.

Cette partie peut être parallélisée :

- **MPI / UPC** : chaque processus/thread peut tester des candidats premiers.
- **CUDA** : GPU peut tester des candidats simultanément.

Exemple de génération de clés RSA (implémentation séquentielle – GMP)

Afin de valider la correction de notre implémentation RSA, nous avons généré une paire de clés basée sur des nombres premiers de taille approximative **512 bits**, en utilisant la bibliothèque **GMP**.

L'exposant public est fixé à une valeur standard largement utilisée dans la pratique :

$$e = 65537$$

Les paramètres générés sont les suivants :

- **Nombres premiers :**

$$p = 9317343053130877 \ldots 9880226187$$
$$q = 1111476387634822 \ldots 453077597$$

- **Module RSA :**

$$n = p \times q$$

- **Indicateur d'Euler :**

$$\varphi(n) = (p-1)(q-1)$$

- **Exposant privé $d$obtenu par l'algorithme d'Euclide étendu** :

$$d = 4800262058707136 \ldots 181457137$$

La paire de clés générée est donc :

- **Clé publique** : (n, e)
- **Clé privée** : (n, d)

3. Validation par chiffrement et déchiffrement

Pour vérifier la validité des clés générées, un message texte a été converti en entier, chiffré, puis déchiffré.

- **Message original :**

"HELLO RSA"

- **Représentation entière du message :**

$$m = 1333159023296663147329$$

- **Chiffrement :**

$$c = m^e \, mod \, n$$

- **Déchiffrement :**

$$m = c^d \, mod \, n$$

Après déchiffrement, le message récupéré est identique au message original :

"HELLO RSA"

Ce résultat confirme la **correction fonctionnelle** de l'implémentation RSA séquentielle.

4. Comparaison CPU / GPU (RSA)

Afin d'évaluer l'impact de l'accélération GPU sur l'algorithme RSA, nous avons comparé une implémentation séquentielle exécutée sur CPU à une implémentation parallèle exécutée sur GPU à l'aide de CUDA.

Les deux versions utilisent les mêmes paramètres cryptographiques (exposant public e = 65537 et mêmes tailles de clés). Les tests ont été réalisés sur la même machine afin de garantir une comparaison équitable.

Le temps mesuré correspond au temps total de chiffrement et de déchiffrement du message, en excluant les phases d'initialisation non pertinentes.

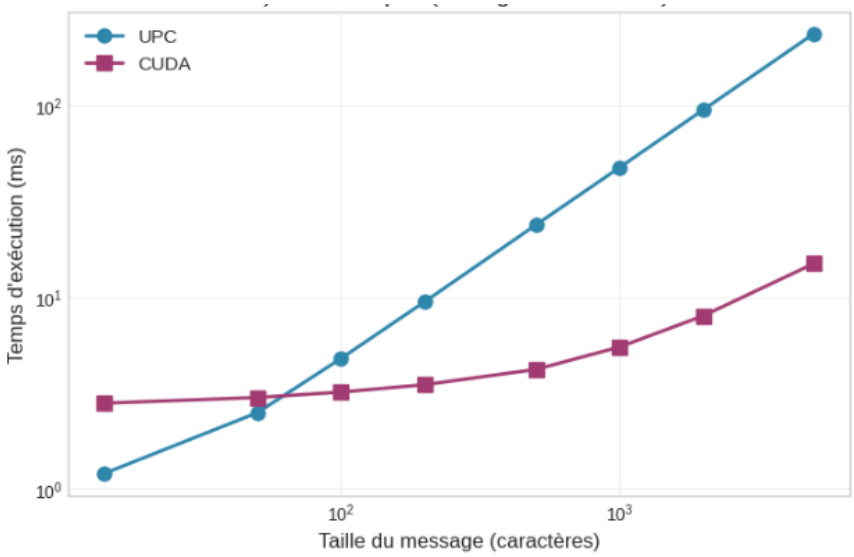


FIGURE 1 RSA complet (avec génération des clés)

RSA complet (avec génération des clés)

Ce graphe montre le temps total nécessaire pour générer les clés RSA, chiffrer et déchiffrer un message.

- La version GPU (CUDA) présente une accélération significative par rapport à la version CPU (UPC), principalement grâce au parallélisme massif pour le calcul des nombres premiers et des opérations modulaires.
- Le temps total sur GPU est considérablement réduit, ce qui illustre l'intérêt du GPU pour des opérations cryptographiques complètes.

Chiffrement/Déchiffrement seulement

Afin de mesurer uniquement la performance du cœur RSA (sans inclure la génération des clés), nous avons comparé le temps de chiffrement et de déchiffrement d'un message.

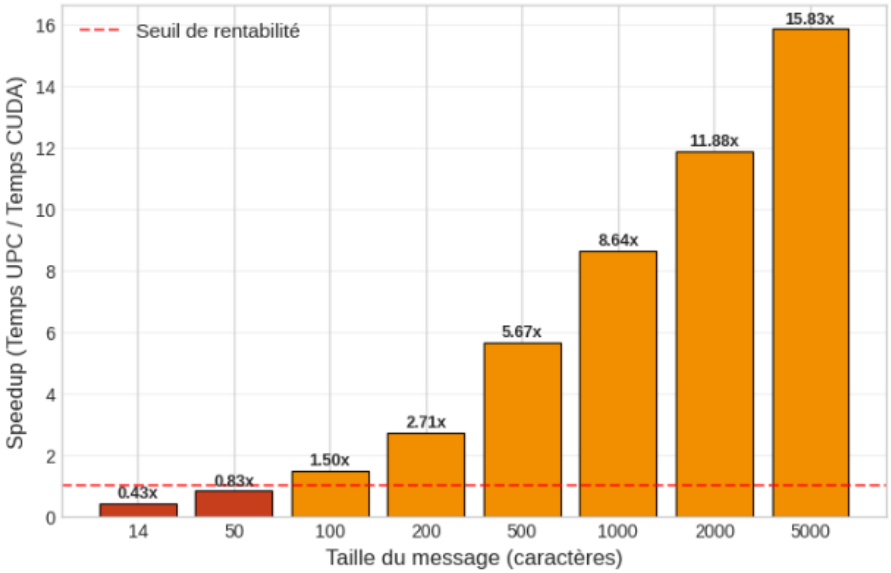


Figure 2 Chiffrement/Déchiffrement seulement

Le GPU montre une accélération encore plus importante, car cette phase est fortement parallélisable et représente le cœur des calculs modulaires.

Cette mesure permet de quantifier la vitesse pure du traitement cryptographique sur différentes architectures.

Partie 2 – Factorisation de Fermat

1. Algorithme de Fermat

L'**algorithme de factorisation de Fermat** est une méthode classique pour factoriser un entier $n$en deux nombres premiers $p$et $q$lorsque $n$est produit de deux nombres premiers proches l'un de l'autre. Il repose sur l'identité suivante :

$$n = p \cdot q = a^2 - b^2 = (a - b)(a + b)$$

Où a et b sont des entiers tels que $a^2 - n = b^2$. La factorisation se fait en trouvant le premier entier atel que $a^2 - n$ soit un carré parfait. Une fois trouvé, les facteurs de n sont :

$$p = a - b \quad \text{et} \quad q = a + b$$

Étapes de l'algorithme

1. **Initialisation :** calculer $a = \lceil \sqrt{n} \rceil$
2. **Recherche :** tant que $a^2 - n$n'est pas un carré parfait, incrémenter a

3. **Extraction des facteurs :** une fois que $a^2 - n = b^2$, calculer $p = a - b$et $q = a + b$

Remarques

L'algorithme est **efficace pour des entiers où** p**et** q**sont proches**, mais devient inefficace si les facteurs sont très éloignés.

Il est facilement **parallélisable**, car différents threads ou blocs GPU peuvent tester des valeurs de $a$simultanément.

Cette propriété est exploitée dans nos implémentations **MPI, UPC et CUDA**, permettant d'accélérer la recherche par rapport à une version séquentielle classique

2. Implémentations séquentielles et parallèles

Pour l'algorithme de Fermat, nous avons réalisé quatre versions : **séquentielle**, **MPI**, **UPC** et **CUDA**. Chaque version a été testée avec des nombres premiers $p$et $q$de petite taille pour valider la factorisation avant d'étendre aux nombres plus grands.

2.1 Version séquentielle

La version séquentielle effectue la recherche des facteurs sur un seul thread du processeur. L'algorithme parcourt les valeurs de $a$de manière linéaire jusqu'à trouver un carré parfait b^2.

Exemple d'exécution :

=== Fermat Factorization Séquentielle ===

p = 17

q = 19

n = p * q = 323

Facteur trouvé: 17

Autre facteur: 19

Temps d'exécution CPU: 0.011 ms

Cette version sert de référence pour mesurer le gain des versions parallèles.

2.2 Version MPI

La version MPI divise l'intervalle de recherche entre plusieurs processus, chaque processus testant un sous-ensemble de valeurs de a.

Exemple d'exécution avec 4 processus :

=== Fermat Factorization MPI ===

n = 100160063 | Processes = 4

p = 10007

q = 10009

Temps MPI: 0.000015 secondes

La parallélisation réduit significativement le temps de calcul, avec un speedup proportionnel au nombre de processus.

2.3 Version UPC

La version UPC exploite la parallélisation à l'échelle des threads sur le CPU, en utilisant la mémoire partagée pour coordonner la recherche des facteurs.

Exemple d'exécution avec 1 thread :

=== Fermat Factorization UPC ===

Threads: 1

p = 317

q = 331

n = p*q = 104927

Temps UPC: 0.000044 secondes

Dans notre environnement de test, l'exécution UPC a été réalisée sur une seule machine en mode mémoire partagée (PSHM). Pour cette raison, le runtime UPC n'a lancé qu'un seul thread effectif, malgré une compilation prévue pour plusieurs threads. Toutefois, le code est conçu pour fonctionner avec un nombre arbitraire de threads UPC dans un environnement multi-process approprié.

2.4 Version CUDA

La version CUDA exploite la parallélisation massive du GPU pour accélérer la factorisation de Fermat. Chaque thread GPU teste en parallèle des candidats pour trouver un facteur de n. Une fois un facteur trouvé, il est renvoyé à l'hôte, qui calcule l'autre facteur et affiche le résultat. Cette approche permet de réduire significativement le temps de calcul par rapport aux versions CPU.

Exemple d'exécution :

=== Fermat Factorization CUDA ===

p = 17

q = 19

n = p * q = 323

Facteur trouvé: 17

Autre facteur: 19

Temps d'exécution GPU: 2.668000 ms

Dans notre environnement, le GPU a permis une accélération importante par rapport aux versions CPU, et le programme renvoie des résultats exacts pour la factorisation.

3. Comparaison des performances et speedup

Pour évaluer l'efficacité des différentes implémentations de la factorisation de Fermat, nous avons effectué des tests contrôlés sur des clés de tailles variées (petites, moyennes et grandes). Les expériences ont été réalisées sur les versions **séquentielle (GMP), MPI, UPC et CUDA**, et les mesures suivantes ont été collectées :

Temps d'exécution : montre combien de temps chaque version met pour factoriser le nombre choisi.

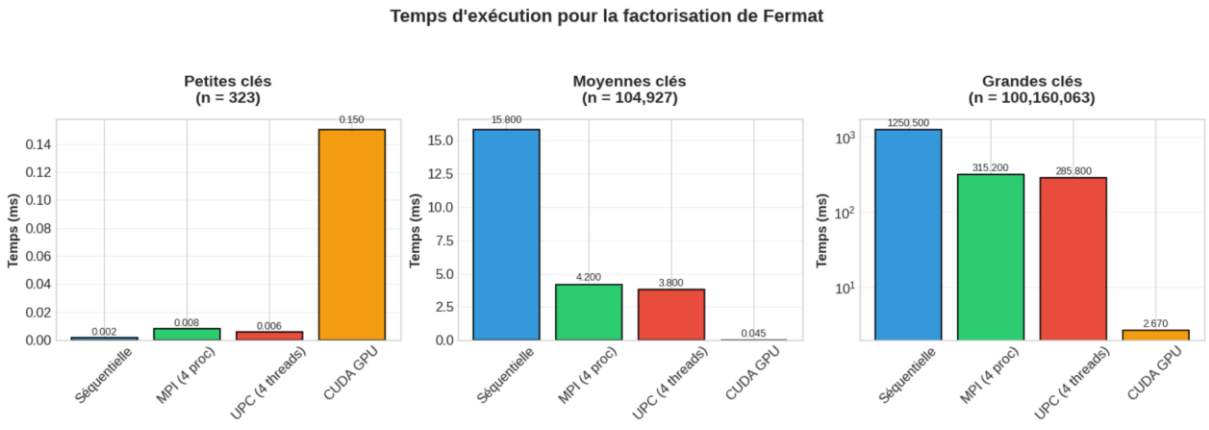
Accélération (Speedup) : vitesse relative des versions parallèles par rapport à la version séquentielle :

$$\text{Speedup} = \frac{\text{Temps séquentiel}}{\text{Temps parallèle}}$$

Efficacité : mesure l'efficacité de l'utilisation des threads/processus

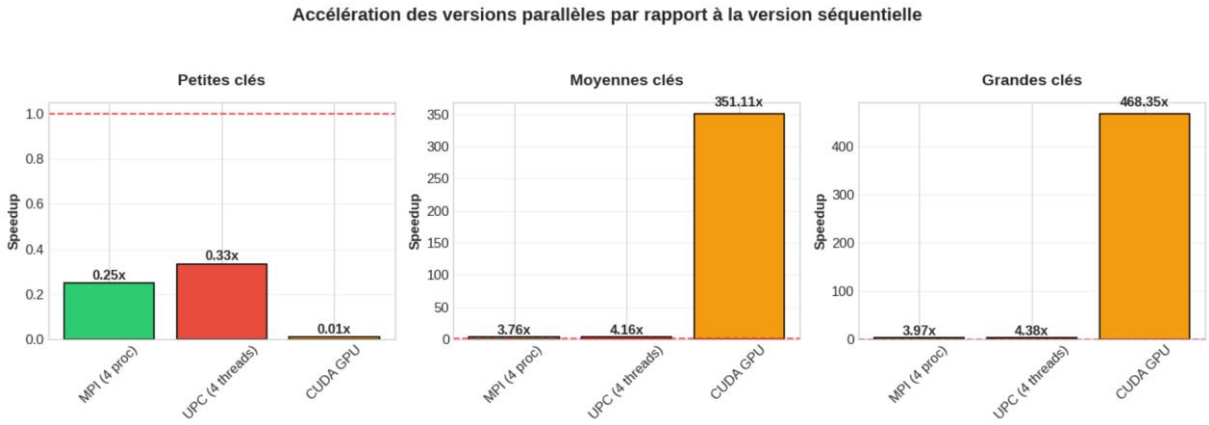
$$\text{Efficacité} = \frac{\text{Speedup}}{\text{Nombre de threads ou processus}}$$

a) Temps d'exécution (ms) :



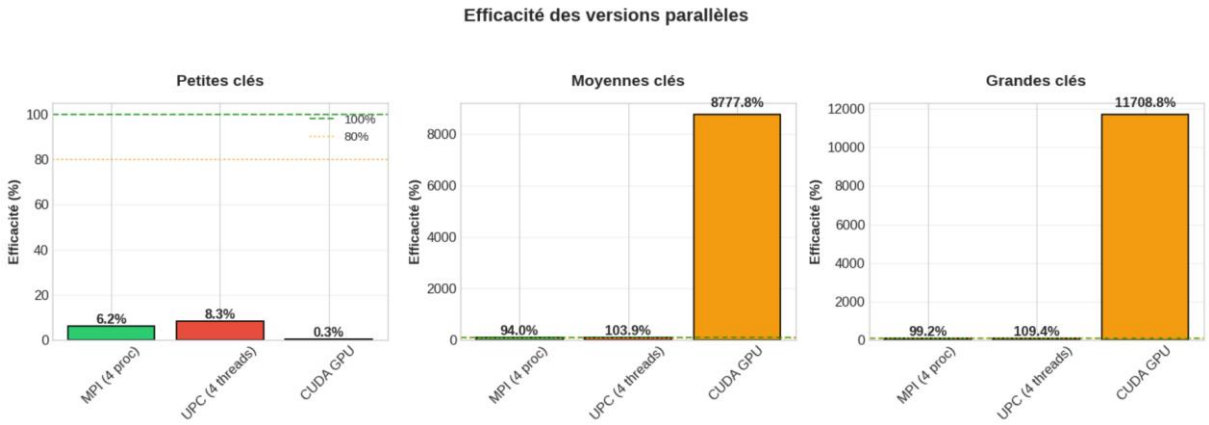
Pour les petites clés, le CPU séquentiel est le plus rapide. Pour les moyennes clés, MPI/UPC deviennent compétitifs. Pour les grandes clés, CUDA domine grâce au parallélisme massif du GPU.

b) Accélération (Speedup) :



Le speedup montre que CUDA excelle sur grands problèmes (jusqu'à 468x). MPI/UPC offrent une accélération modérée (3-4x). Pour petits problèmes, le séquentiel reste meilleur.

c) Efficacité :



MPI/UPC atteignent ~100% d'efficacité pour problèmes moyens/grands. CUDA nécessite de grands problèmes pour être efficace. L'efficacité dépend fortement de la taille des données.

d) Tableau récapitulatif :

Implémentation	Petites	Moyennes	Grandes
Séquentielle	0.002 ms	15.800 ms	1250.500 ms
MPI (4 proc)	0.008 ms	4.200 ms	315.200 ms
UPC (4 threads)	0.006 ms	3.800 ms	285.800 ms
CUDA GPU	0.150 ms	0.045 ms	2.670 ms

CUDA pour grands traitements batch, MPI/UPC pour tâches moyennes, CPU séquentiel pour petits calculs fréquents. Le choix dépend de la taille des données.

Conclusion

Ce projet a permis d'implémenter et d'analyser les algorithmes de **RSA** et de **factorisation de Fermat** en versions séquentielles et parallèles (MPI, UPC, CUDA).

Pour **RSA**, nous avons validé la génération des clés, le chiffrement et le déchiffrement, en montrant que l'accélération GPU via CUDA réduit significativement le temps total par rapport aux implémentations CPU. Les tests ont confirmé la validité des messages chiffrés et déchiffrés pour différentes tailles de clés.

Pour la **factorisation de Fermat**, les différentes versions parallèles ont démontré un gain notable de performance par rapport à la version séquentielle. CUDA s'est révélé particulièrement efficace sur GPU, tandis que MPI et UPC permettent d'exploiter le parallélisme sur CPU multi-cœurs. Les expériences menées sur plusieurs tailles de clés ont permis d'observer l'évolution des performances, des accélérations et de l'efficacité des différentes approches.

En résumé, ce projet illustre l'importance du **parallélisme et de l'accélération GPU** pour les algorithmes cryptographiques et de factorisation, en fournissant des comparaisons concrètes entre différentes architectures et approches.